

INFORME DE CONTRIBUCIÓN PERSONAL

Proyecto: Pico Botella (PB) — Android / Kotlin

Repositorio: <https://github.com/StiviMoon/PB> | Versión: 1.1.0

Historias de Usuario asignadas: HU 9.0 (Eliminar Reto) y HU 12 (Mostrar Reto Aleatorio)

Este documento detalla de forma exhaustiva el trabajo que realicé en el proyecto grupal **Pico Botella**. Se explica criterio por criterio qué se hizo, qué herramientas y componentes de Android se utilizaron, y cómo se escribió el código para cumplir con cada requerimiento de las dos historias de usuario que me fueron asignadas.

1. Contexto Técnico del Proyecto

La aplicación sigue el patrón de arquitectura **MVVM (Model-View-ViewModel)**, el estándar de Android moderno. Esto significa que el código está separado en tres capas:

- ✓ **Vista (Fragment/Dialog):** Solo muestra datos y captura las acciones del usuario.
- ✓ **ViewModel:** Contiene la lógica de negocio (insertar, editar, eliminar retos). No depende de la UI.
- ✓ **Repository + Room:** Gestiona la base de datos SQLite local. Cuando los datos cambian, notifica automáticamente a la Vista.

Herramientas y librerías que afectan mi trabajo:

Herramienta / Librería	Para qué la usé en mis HUs
Room (SQLite)	Para leer y eliminar retos de la base de datos local
Kotlin Coroutines	Para hacer operaciones de BD en segundo plano sin bloquear la pantalla
StateFlow / Flow	Para que la lista de retos se actualice automáticamente al eliminar uno
android.app.Dialog	Para construir los cuadros de diálogo personalizados de HU 9.0 y HU 12
HttpURLConnection	Para descargar el JSON de la API de Pokémon (HU 12)
Dispatchers.IO	Para ejecutar la descarga de la imagen del Pokémon en hilo secundario
ViewGroup.LayoutParams	Para controlar el tamaño del diálogo a pantalla completa (HU 12)
ConstraintLayout	Usado en dialog_random_reto.xml para posicionar el botón sobresaliente
Shape Drawables (XML)	Para crear el fondo oscuro redondeado y el círculo del Pokémon

2. HU 9.0 — Cuadro de Diálogo: Eliminar Reto

Historia de Usuario: Como App, quiero poder mostrarle al jugador un cuadro de diálogo para que el jugador elimine un determinado reto.

Campo	Valor
ID	HU 9.0

Campo	Valor
Actor	App
Acción	Poder mostrarle al jugador un cuadro de diálogo
Consecuencia	Que el jugador elimine un determinado reto
Archivos creados/modificados	RetosFragment.kt dialog_delete_reto.xml RetosViewModel.kt

2.1. Criterios de Aceptación — Implementación Detallada

■ Criterio 1: El cuadro de diálogo debe tener un fondo de color blanco.

¿Qué hice?

Creé el archivo de diseño **dialog_delete_reto.xml**. El componente raíz es un **LinearLayout** al que le asigné el atributo **android:background** apuntando al recurso drawable **bg_dialog_white_rounded.xml**, que define un rectángulo blanco con esquinas redondeadas. En el código Kotlin, al crear el Dialog, configuré la ventana para que su fondo sea transparente, de modo que solo se vea el fondo blanco del layout y no el fondo gris predeterminado de Android:

```
// RetosFragment.kt – función confirmarEliminar()
val dialog = android.app.Dialog(requireContext())
dialog setContentView(R.layout.dialog_delete_reto)
dialog.window?.setBackgroundDrawable(
    android.graphics.drawable.ColorDrawable(android.graphics.Color.TRANSPARENT)
)
```

■ Criterio 2: Título en la parte superior con el texto "¿Desea eliminar el siguiente reto?", centrado y en negro (bold).

¿Qué hice?

En el **dialog_delete_reto.xml** coloqué un **TextView** como primer hijo del **LinearLayout** con los siguientes atributos:

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="¿Desea eliminar el siguiente reto?:"
    android:textColor="#000000"          <!-- negro puro -->
    android:textStyle="bold"             <!-- negrita -->
    android:textSize="18sp"
    android:textAlignment="center"       <!-- centrado -->
    android:layout_marginBottom="16dp"/>
```

■ Criterio 3: Mostrar la descripción del reto que se quiere eliminar (información viene de la base de datos).

¿Qué hice?

En el XML coloqué un segundo **TextView** con id **tvRetoDescription**. En el código Kotlin, dentro de la función **confirmarEliminar(reto: Reto)** del **RetosFragment**, accedí a ese **TextView** y le asigné el texto del objeto Reto que ya viene cargado de la base de datos:

```
// El objeto 'reto' ya viene de la BD cuando el usuario pulsa eliminar
val tvRetoDescription = dialog.findViewById<TextView>(R.id.tvRetoDescription)
tvRetoDescription.text = reto.descripcion // ← descripción real de la BD
```

■ Criterio 4: Texto "NO" de color naranja. Al hacer clic cierra el diálogo y vuelve a la lista de retos.

¿Qué hice?

Usé un **TextView** (no un Button) con id **btnNo** y le asigné el color naranja **#FF6F00**. El componente es clickeable y tiene efecto visual de ripple. En Kotlin le programé el listener para simplemente cerrar el diálogo:

```
<!-- dialog_delete_reto.xml -->
<TextView
    android:id="@+id/btnNo"
    android:text="NO"
    android:textColor="#FF6F00"
    android:textStyle="bold"
    android:clickable="true"
    android:focusable="true"
    android:background="?attr/selectableItemBackgroundBorderless" />

// RetosFragment.kt
val btnNo = dialog.findViewById<TextView>(R.id.btnNo)
btnNo.setOnClickListener {
    dialog.dismiss() // solo cierra el diálogo, no borra nada
}
```

■ **Criterio 5: Texto "SI" de color naranja. Al hacer clic, elimina el reto de SQLite e inmediatamente desaparece de la lista.**

¿Qué hice?

Al igual que el NO, usé un **TextView** con id **btnSi** y color naranja. En su listener llamé al **ViewModel** para que ejecute la eliminación. Gracias al patrón MVVM y al **Flow** reactivo de Room, la lista se actualiza sola sin necesidad de recargar la pantalla:

```
// RetosFragment.kt
val btnSi = dialog.findViewById<TextView>(R.id.btnSi)
btnSi.setOnClickListener {
    viewModel.eliminarReto(reto) // ← llama al ViewModel
    dialog.dismiss()
}

// RetosViewModel.kt
fun eliminarReto(reto: Reto) {
    viewModelScope.launch { // Coroutine – hilo secundario
        repository.deleteReto(reto) // ejecuta DELETE en SQLite
    }
}

// RetoRepository.kt
suspend fun deleteReto(reto: Reto) = dao.delete(reto)

// RetoDao.kt
@Delete
suspend fun delete(reto: Reto) // Room genera el SQL automáticamente
```

Flujo completo al presionar SI: **Vista** → **ViewModel** → **Repository** → **DAO** → **SQLite**. Room detecta el cambio, el Flow emite la nueva lista sin ese reto, el ViewModel actualiza el StateFlow, y el Fragment refresca el RecyclerView. Todo esto ocurre en menos de un segundo.

■ **Criterio 6: El diálogo solo desaparece al dar clic en NO o SI. Al tocar fuera del diálogo NO debe cerrarse.**

¿Qué hice?

Esta es una propiedad específica de **android.app.Dialog**. Llamé al método **setCancelable(false)** justo después de crear el diálogo. Esto instruye al sistema operativo Android para que ignore todos los toques fuera del área del diálogo y también que no se cierre con el botón de atrás del dispositivo:

```
val dialog = android.app.Dialog(requireContext())
dialog setContentView(R.layout.dialog_delete_reto)
dialog.window?.setBackgroundDrawable(ColorDrawable(Color.TRANSPARENT))
dialog.setCancelable(false) // ← Criterio 6: fuerza al usuario a elegir SI o NO
// ... configuración de botones ...
dialog.show()
```

2.2. Resumen de Archivos que Creé/Modifiqué para HU 9.0

Archivo	Tipo	Qué hice
dialog_delete_reto.xml	Layout XML	Creé el diseño del diálogo: fondo blanco, título, descripción dinámica, botones NO y SI en na
bg_dialog_white_rounded.xml	Drawable XML	Definé el shape blanco con esquinas redondeadas para el fondo del diálogo
RetosFragment.kt	Kotlin	Añadí la función confirmarEliminar() que infla el diálogo, conecta la descripción del reto y ges
RetosViewModel.kt	Kotlin	Ya existía; usé la función eliminarReto(reto) que ejecuta el delete en Room mediante Corout

3. HU 12 — Mostrar Reto Aleatorio

Historia de Usuario: Como App, quiero mostrar un cuadro de diálogo al jugador para que pueda conocer el reto aleatorio que debe realizar un determinado jugador.

Campo	Valor
ID	HU 12
Actor	App
Acción	Mostrar un cuadro de diálogo al jugador
Consecuencia	Conocer el reto aleatorio que debe realizar un determinado jugador
Archivos creados/modificados	HomeFragment.kt dialog_random_reto.xml PokemonRepository.kt bg_dialog_rounded.xml bg_circle_pokemon.xml

3.1. Criterios de Aceptación — Implementación Detallada

■ **Criterio 1: Cuadro de diálogo personalizado con fondo negro degradado, transparencia sutil y bordes redondeados de color blanco.**

¿Qué hice?

Creé el archivo **dialog_random_reto.xml** usando un **ConstraintLayout** externo con fondo transparente, y un **ConstraintLayout** interno como contenedor del diálogo con el atributo **android:background="@drawable/bg_dialog_rounded"**. Este drawable es un shape XML que define el fondo oscuro con transparencia y el borde blanco:

```
<!-- bg_dialog_rounded.xml -->
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#CC000000"/>           <!-- negro con ~80% opacidad -->
    <stroke android:width="2dp" android:color="#FFFFFF"/>    <!-- borde blanco -->
    <corners android:radius="16dp"/>           <!-- esquinas redondeadas -->
</shape>
```

En Kotlin, al crear el Dialog configuré la ventana para que ocupe el ancho completo de la pantalla, para que el diálogo se vea centrado y grande:

```
val dialog = Dialog(requireContext())
dialog setContentView(R.layout.dialog_random_reto)
dialog.window?.setBackgroundDrawable(ColorDrawable(Color.TRANSPARENT))
dialog.window?.setLayout(
    ViewGroup.LayoutParams.MATCH_PARENT,    // ancho: toda la pantalla
    ViewGroup.LayoutParams.WRAP_CONTENT    // alto: lo que necesite el contenido
)
```

■ **Criterio 2: Círculo con borde blanco y fondo negro en la parte superior. Mostrar imagen aleatoria de un Pokémon consumiendo la API.**

¿Qué hice?

En el **dialog_random_reto.xml** posicioné un **ImageView** con id **ivPokemon** usando constraints del **ConstraintLayout** para que esté anclado entre el borde superior del diálogo y el borde inferior del mismo, logrando el efecto de que 'sobresale' en la parte superior. Le asigné el background **bg_circle_pokemon.xml**:

```
<!-- bg_circle_pokemon.xml -->
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">           <!-- forma circular perfecta -->
    <solid android:color="#000000"/>
```

```

        <stroke android:width="3dp" android:color="#FFFFFF"/>
    </shape>

<!-- dialog_random_reto.xml -->
<ImageView
    android:id="@+id/ivPokemon"
    android:layout_width="80dp"
    android:layout_height="80dp"
    android:background="@drawable/bg_circle_pokemon"
    android:padding="12dp"
    android:scaleType="fitCenter"
    app:layout_constraintTop_toTopOf="@id/dialogContainer"
    app:layout_constraintBottom_toTopOf="@id/dialogContainer"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"/>

```

Para obtener la imagen aleatoria del Pokémon, creé la clase **PokemonRepository.kt** que realiza la petición HTTP a la API especificada. La petición corre en **Dispatchers.IO** (hilo de red) para no bloquear la interfaz:

```

// PokemonRepository.kt
class PokemonRepository {
    suspend fun getRandomPokemonImageUrl(): String? = withContext(Dispatchers.IO) {
        val url = URL("https://raw.githubusercontent.com/Biuni/PokemonGO-Pokedex/master/pokedex.json")
        val connection = url.openConnection() as HttpURLConnection
        connection.requestMethod = "GET"

        if (connection.responseCode == HttpURLConnection.HTTP_OK) {
            val json = connection.inputStream.bufferedReader().use { it.readText() }
            val pokemonArray = JSONObject(json).getJSONArray("pokemon")
            val randomPokemon = pokemonArray.getJSONObject(Random.nextInt(pokemonArray.length()))
            var imageUrl = randomPokemon.getString("img")
            // Android no permite http:// en versiones modernas, se reemplaza por https://
            if (imageUrl.startsWith("http://"))
                imageUrl = imageUrl.replace("http://", "https://")
            return@withContext imageUrl
        }
        return@withContext null
    }
}

```

La imagen se descarga DESPUÉS de mostrar el diálogo (en una Coroutine separada) para que el jugador vea el reto inmediatamente y la imagen aparezca en cuanto cargue:

```

// HomeFragment.kt – dentro de mostrarRetoAleatorio()
dialog.show() // ← primero se muestra el diálogo con el texto del reto

launch { // ← luego, en paralelo, se descarga la imagen
    val imageUrl = PokemonRepository().getRandomPokemonImageUrl()
    imageUrl?.let {
        val bitmap = withContext(Dispatchers.IO) {
            val conn = URL(it).openConnection() as HttpURLConnection
            conn.connectTimeout = 5000; conn.readTimeout = 5000; conn.connect()
            BitmapFactory.decodeStream(conn.inputStream)
        }
        if (dialog.isShowing) { // solo si el diálogo sigue abierto
            dialog.findViewById<ImageView>(R.id.ivPokemon).setImageBitmap(bitmap)
        }
    }
}

```

```
}  
}
```

■ **Criterio 3: Texto de color blanco (bold) que muestra el reto a realizar. El reto debe venir de la base de datos local y debe ser aleatorio.**

¿Qué hice?

En el XML coloqué un **TextView** con id **tvReto**, color blanco y estilo bold. En el código, consulté la base de datos local usando el método **getRandomReto()** del DAO, que ejecuta la consulta **SELECT * FROM retos ORDER BY RANDOM() LIMIT 1**:

```
<!-- dialog_random_reto.xml -->  
<TextView  
    android:id="@+id/tvReto"  
    android:textColor="#FFFFFF"  
    android:textSize="18sp"  
    android:textStyle="bold"  
    android:textAlignment="center"/>  
  
// HomeFragment.kt  
val dao = AppDatabase.getInstance(requireContext()).retoDao()  
val reto = dao.getRandomReto()           // ← consulta aleatoria a la BD  
val retoTexto = reto?.descripcion ?: "No hay retos disponibles."  
dialog.findViewById<TextView>(R.id.tvReto).text = retoTexto  
  
// RetoDao.kt – la consulta SQL que usé  
@Query("SELECT * FROM retos ORDER BY RANDOM() LIMIT 1")  
suspend fun getRandomReto(): Reto?
```

■ **Criterio 4: Botón naranja "Cerrar" en la parte inferior-central. La parte superior dentro del diálogo, la inferior por fuera.**

¿Qué hice?

Este fue el reto visual más interesante. Usé un **AppCompatButton** con el background **btn_orange_rounded.xml** (shape con relleno naranja y esquinas redondeadas). Para lograr que la parte superior del botón quede DENTRO del diálogo y la inferior FUERA, usé las restricciones de **ConstraintLayout** anclando simultáneamente el borde superior e inferior del botón al borde inferior del contenedor del diálogo:

```
<!-- dialog_random_reto.xml – posicionamiento del botón Cerrar -->  
<androidx.appcompat.widget.AppCompatButton  
    android:id="@+id/btnCerrar"  
    android:background="@drawable/btn_orange_rounded"  
    android:text="Cerrar"  
    android:textColor="#FFFFFF"  
    android:paddingHorizontal="32dp"  
    app:layout_constraintTop_toBottomOf="@id/dialogContainer"    <!-- top = borde inferior del diálogo -->  
    app:layout_constraintBottom_toBottomOf="@id/dialogContainer"<!-- bottom = mismo punto -->  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintEnd_toEndOf="parent"/>  
  
<!-- Al coincidir Top y Bottom en el mismo borde, el botón queda centrado en ese punto,  
    con la mitad superior dentro del diálogo y la mitad inferior fuera -->
```

■ **Criterio 5: Al hacer clic en "Cerrar", el diálogo desaparece y el juego vuelve a la pantalla principal listo para nueva partida.**

¿Qué hice?

Al botón de cerrar le asigné un listener que: (1) oculta el diálogo con **dialog.dismiss()**, (2) hace visible de nuevo el botón de girar la botella, (3) reanuda la música de fondo si el audio estaba activado:

```
// HomeFragment.kt
dialog.findViewById<Button>(R.id.btnCerrar).setOnClickListener {
    dialog.dismiss()           // cierra el diálogo
    resetButton()             // muestra de nuevo el botón "Presióname" con animación
    audioViewModel.resumeIfEnabled() // reanuda la música si estaba activa
}

private fun resetButton() {
    binding.btnPress.visibility = View.VISIBLE
    binding.btnPress.startAnim(R.anim.btn_blink) // vuelve la animación parpadeante
}
```

■ **Criterio 6: El diálogo solo desaparece al hacer clic en el botón 'Cerrar'. Al tocar por fuera NO debe cerrarse.**

¿Qué hice?

Exactamente igual que en la HU 9.0, llamé al método **setCancelable(false)** del Dialog. Esto garantiza que: (a) los toques fuera del diálogo son ignorados, y (b) el botón de atrás del teléfono tampoco lo cierra. La ÚNICA forma de cerrar el diálogo es presionar el botón naranja 'Cerrar':

```
val dialog = Dialog(requireContext())
dialog setContentView(R.layout.dialog_random_reto)
dialog.window?.setBackgroundDrawable(ColorDrawable(Color.TRANSPARENT))
dialog.window?.setLayout(MATCH_PARENT, WRAP_CONTENT)
dialog.setCancelable(false) // ← Criterio 6
// ... configuración ...
dialog.show()
```

3.2. Resumen de Archivos que Creé/Modifiqué para HU 12

Archivo	Tipo	Qué hice
dialog_random_reto.xml	Layout XML	Diseñé el diálogo completo: contenedor oscuro, ImageView circular, texto del reto en blanco
bg_dialog_rounded.xml	Drawable XML	Shape negro semitransparente con borde blanco y esquinas redondeadas (fondo del diálogo)
bg_circle_pokemon.xml	Drawable XML	Shape oval negro con borde blanco (fondo circular del ImageView del Pokémon)
btn_orange_rounded.xml	Drawable XML	Shape naranja con esquinas redondeadas para el botón Cerrar
PokemonRepository.kt	Kotlin	Creé esta clase. Hace la petición HTTP a la API de Pokémon, parsea el JSON y retorna una lista
HomeFragment.kt	Kotlin	Añadí la función mostrarRetoAleatorio(): consulta la BD, crea el diálogo, carga la imagen en el ImageView
RetoDao.kt	Kotlin	Añadí la función getRandomReto() con la consulta SQL ORDER BY RANDOM() LIMIT 1

4. Conclusión

Las dos Historias de Usuario que me fueron asignadas (**HU 9.0** y **HU 12**) fueron implementadas cumpliendo al 100% con todos y cada uno de sus Criterios de Aceptación. A continuación, el resumen de tecnologías y decisiones de diseño que tomé:

Decisión Técnica	Justificación
Usar android.app.Dialog en lugar de DialogFragment	Permite controlar directamente el layout y el comportamiento del cuadro de forma directa desde el Fragment

Decisión Técnica	Justificación
setCancelable(false) en ambos diálogos	Requerimiento explícito del Criterio 6 de ambas HUs. Obliga al jugador a tomar una decisión consciente.
Coroutine separada para descargar la imagen (ifAsync)	En (ifAsync) se muestra inmediatamente con el reto. La imagen del Pokémon aparece en cuanto carga, sin
ConstraintLayout con constraints duales en el botón de reposicionamiento	El botón de reposicionamiento que logra el efecto visual de que el botón 'sobresale' del borde inferior del d
HttpURLConnection nativo (sin librerías externas)	El proyecto no incluye Retrofit ni Glide, así que usé la API de red nativa de Java/Kotlin para no agregar c
Sustitución http:// → https:// en URLs de la API	Android bloquea tráfico HTTP en claro por defecto (cleartext restriction). Esta corrección permite que las